Ain Shams University
Faculty of Engineering
Electronics and Communications Eng. Dept.

ECE 352s: Analog and Digital Communication Systems
Spring 2026

Course Project

Gram-Schmidt Orthogonalization, Constellation Diagrams, and BER Analysis

Team 11 Members:

Youssef Medhat Ali	2300628
Khaled Abdelrahman Hussien	2300260
Abdelrahman Mohamed Aly	2300518
Mostafa Mohamed Abdullah	2300565
Ganna Abdelhamid Mohamed	2300277
Amr Hatem Sabry	2300729
Abdelrahman Sherif Medhat	2300100
Ali Amir Abouzaid	2300157
Mohamed Fouad Mahmoud	2300442

Submitted to:

Prof. Bassant Abdelhamid
TA: Amr Hammouda

Team Contribution Table

No.	Student Name	Contribution
1	Yossef Medhat Ali	Part 1 and Part 2
2	Khaled Abdelrahman Hussien	Report and Part 1
3	Abdelrahman Mohamed Aly	Part 2
4	Mostafa Mohamed Abdullah	Part 1
5	Ganna Abdelhamid Mohamed	Part 1
6	Amr Hatem Sabry	Part 2
7	Abdelrahman Sherif Medhat	Part 1 and Part 2
8	Ali Amir Abouzaid	GUI Development & Report
9	Mohamed Fouad Mahmoud	GUI Integration & and Part 1

Contents

Team Contribution Table **1**

Part 1: Gram-Schmidt Procedure and Signal Representation **3**
Interactive GUI Integration . . . 3
MATLAB Implementation . . . 3
Simulation Results and Energy Calculations . . . 6
Signal Set 1 . . . 6
Signal Set 2 . . . 8
Signal Set 3 . . . 10

Part 2: Error Performance over AWGN Channel **12**
Interactive GUI Integration . . . 12
MATLAB Implementation . . . 12
Theoretical Derivations . . . 16
1. Unipolar NRZ . . . 16
2. Manchester Encoding . . . 16
Simulation Results and comments . . . 16
Commentary on the Results . . . 19

Part 1: Gram-Schmidt Procedure and Signal Representation

Interactive GUI Integration

The implementation for this section features a fully integrated Graphical User Interface (GUI) driven by consecutive dialog boxes and menus. Rather than hard-coding signals, the user is interactively prompted to construct M arbitrary signals segment-by-segment (DC, Sinusoidal, Exponential, or Custom Equations), setting the global simulation time and step dynamically.

MATLAB Implementation

```
clc; clear; close all;

%%MINI GUI
function GramSchmidt_GUI()

    fig = uifigure('Name', 'Gram-Schmidt & Constellation GUI', 'Position', [100 100 500 550]);


    uilabel(fig, 'Position', [20 500 250 22], 'Text', 'Max time duration of signals (seconds):');
    time_edit = uieditfield(fig, 'numeric', 'Position', [280 500 100 22], 'Value', 4);

    uilabel(fig, 'Position', [20 460 250 22], 'Text', 'Number of signals (M):');
    num_edit = uieditfield(fig, 'numeric', 'Position', [280 460 100 22], 'Value', 4);

    uilabel(fig, 'Position', [20 420 450 22], 'Text', 'Signal expressions (one per line):');
    expr_area = uitextarea(fig, 'Position', [20 280 450 130], ...
        'Value', { ...
            '2.*( t<1) - 1.*(t>=1 & t<4)'; ...
            'sin(2*pi*t).*(t<2.5) + cos(2*pi*t).*(t>=2.5)'; ...
            '4.*(t<1) - 3.*(t>=1 & t<2) + 1.*(t>=2)'; ...
            'exp(t)' ...
        });


    btn = uibutton(fig, 'push', 'Position', [175 230 150 35], ...
        'Text', 'Calculate & Plot', 'FontWeight', 'bold');


    uilabel(fig, 'Position', [20 190 200 22], 'Text', 'Energy Calculations:');
    result_area = uitextarea(fig, 'Position', [20 20 450 160], 'Editable', 'off');

    btn.ButtonPushedFcn = @(btn,event) run_calculations(time_edit, num_edit, expr_area, result_area);
end

function run_calculations(time_edit, num_edit, expr_area, result_area)
    No_sym = time_edit.Value;
    M = num_edit.Value;
    exprs = expr_area.Value;

    if length(exprs) < M, uialert(time_edit.Parent, 'Provide enough signal expressions.', 'Error'); return; end

    ts = 0.001;
    t = 0: ts: No_sym - ts;
    L = length(t);
    signals_matrix = zeros(M, L);

    for i = 1:M
        try
            expr = exprs{i};
            signals_matrix(i,:) = eval(expr);
        catch ME
```

```
52              uialert(time_edit.Parent, sprintf('Error in signal %d: %s', i, ME.message), '
                    Error'); return;
53          end
54      end
55
56      my_plot(signals_matrix,'s');
57      [basis_func,basis_coeff] = gram_schmidt(signals_matrix,ts);
58      my_plot(basis_func,'phi');
59
60      result_text = {};
61      for i = 1:M
62          E_time = energy(signals_matrix(i,:), ts);
63          E_const = energy_constellation(basis_coeff(i,:));
64
65          result_text{end+1} = sprintf('--- Signal %d ---', i);
66          result_text{end+1} = sprintf('Energy (Time Domain): %f', E_time);
67          result_text{end+1} = sprintf('Energy (Constellation): %f', E_const);
68          result_text{end+1} = '';
69      end
70      result_area.Value = result_text;
71
72      const_diag(basis_coeff);
73  end
74
75  % ========================================================================
76  %                 CONSTELLATION FUNCTION
77
78  function const_diag(basis_coeff)
79      [M, N] = size(basis_coeff);
80      figure('Name', 'Constellation Diagram', 'NumberTitle','off', 'Color',[1 1 1]);
81      colors_c = lines(M);
82      hold on; grid on;
83
84      if N == 1
85          for m = 1 : M
86              scatter(basis_coeff(m,1), 0, 120, colors_c(m,:), 'filled');
87              text(basis_coeff(m,1), 0.08, sprintf('s_{%d}',m), 'HorizontalAlignment', '
                    center', 'Color', colors_c(m,:), 'FontWeight', 'bold');
88          end
89          xlabel('\phi_1'); ylim([-1 1]);
90          xline(0,'k--');
91      elseif N == 2
92          for m = 1 : M
93              scatter(basis_coeff(m,1), basis_coeff(m,2), 120, colors_c(m,:), 'filled');
94              text(basis_coeff(m,1) + 0.05, basis_coeff(m,2) + 0.05, sprintf('s_{%d}',m), '
                    Color', colors_c(m,:), 'FontWeight', 'bold');
95          end
96          xlabel('\phi_1'); ylabel('\phi_2');
97          xline(0,'k--'); yline(0,'k--');
98      else
99          for m = 1 : M
100             scatter3(basis_coeff(m,1), basis_coeff(m,2), basis_coeff(m,3), 120, colors_c(
                    m,:), 'filled');
101             text(basis_coeff(m,1), basis_coeff(m,2), basis_coeff(m,3), sprintf('  s_{%d}'
                    ,m), 'Color', colors_c(m,:), 'FontWeight', 'bold');
102         end
103         xlabel('\phi_1'); ylabel('\phi_2'); zlabel('\phi_3');
104         view(35, 25);
105     end
106     title(sprintf('Constellation Diagram (%d-D)', min(N, 3)), 'FontSize', 12);
107 end
108
109 % ========================================================================
110 %                   GRAM SCHMIDT FUNCTION
111
112
113 function [basis_functions,basis_coeff] = gram_schmidt(signals_matrix,ts)
114     M = size(signals_matrix, 1);
115     L = size(signals_matrix, 2);
116      basis_functions = zeros(M,L);
117      basis_coeff = zeros(M);
118     E_s1 = energy(signals_matrix(1,:),ts);
119     basis_functions(1,:) = signals_matrix(1,:)/sqrt(E_s1);
```

```
    basis_coeff(1) = sqrt(E_s1);
    N = 1;
    for i = 2:M
        for j = 1:N
                basis_coeff(i,j) = sum(basis_functions(j,:).*signals_matrix(i,:))*ts;
       end
        g_i = signals_matrix(i,:) - basis_coeff(i,1:N)*basis_functions(1:N,:);
        g_i(abs(g_i)<1e-10) = 0;
        if(max(abs(g_i)) < 1e-10)
           continue;
        else
            N = N+1;
            E_gi = energy(g_i,ts);
            basis_functions(N,:) = (g_i)/sqrt(E_gi);
            basis_coeff(i,N) = sqrt(E_gi);
       end
    end
    basis_functions = basis_functions(1:N,:);
     basis_coeff = basis_coeff(:,1:N);
end
% ========================================================================
%                  MY PLOT FUNCTION

function my_plot(signals_matrix, symbol_str)
    M = size(signals_matrix, 1);
     ts = 0.001;
    t = 0:ts:(size(signals_matrix, 2)-1)*ts;
    num_rows = ceil(sqrt(M)); num_cols = ceil(M / num_rows);
    main_fig = figure('Name', symbol_str); set(main_fig, 'color', 'w');
    for i = 1:M
        ax = subplot(num_rows, num_cols, i);
        plot(ax, t, signals_matrix(i, :), 'b', 'LineWidth', 1.5);
        grid(ax, 'on'); ax.Box = 'on'; ax.XColor = [0 0 0]; ax.YColor = [0 0 0];
        if(symbol_str == "phi"), title_str = sprintf('$\\%s_{%d}(t)$', symbol_str, i);
        else, title_str = sprintf('$%s_{%d}(t)$', symbol_str, i); end
        title(ax, title_str, 'Interpreter', 'latex', 'FontSize', 12);
        xlabel(ax, 'Time (s)'); ylabel(ax, 'Amplitude');
        ylim(ax, [min(signals_matrix(:))-0.5, max(signals_matrix(:))+0.5]);
    end
end
% ========================================================================
%                  ENERGY USING TIME DOMAIN FUNCTION
function E = energy(signal,ts), E = sum(abs(signal).^2)*ts; end

% ========================================================================
%                  ENERGY USING CONSTELLATION FUNCTION

function E = energy_constellation(basis_coeff), E = sum(basis_coeff.^2);
end
```

Simulation Results and Energy Calculations

Signal Set 1

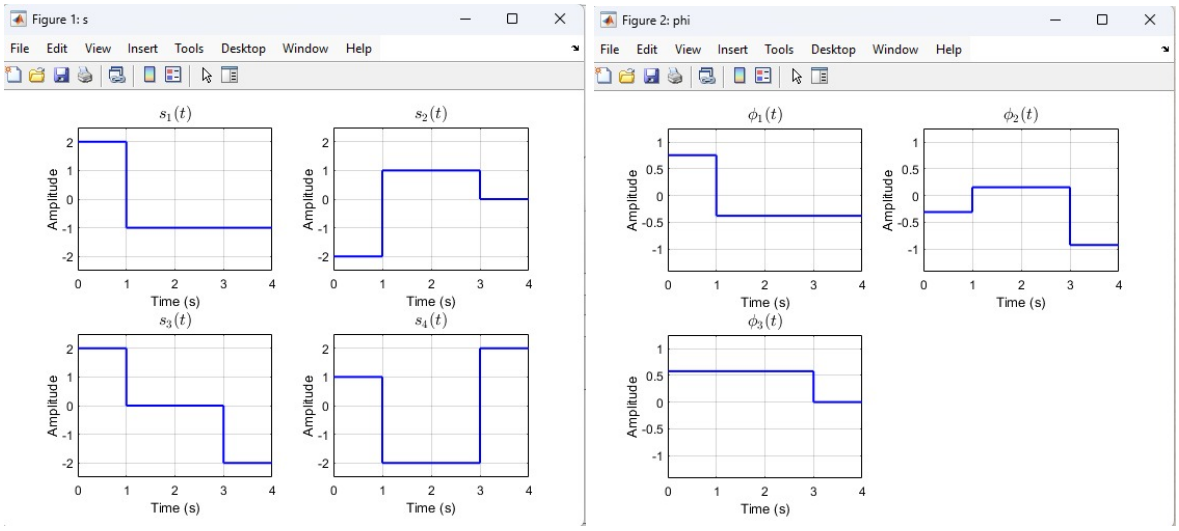
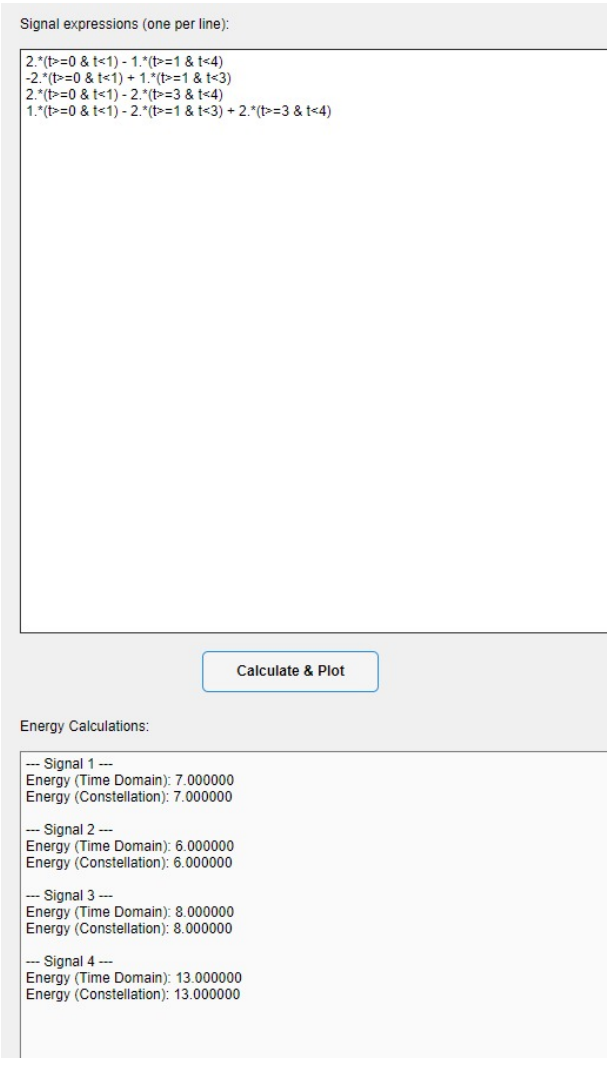


Figure 1: Input Signals and Gram-Schmidt Basis Functions for Set 1



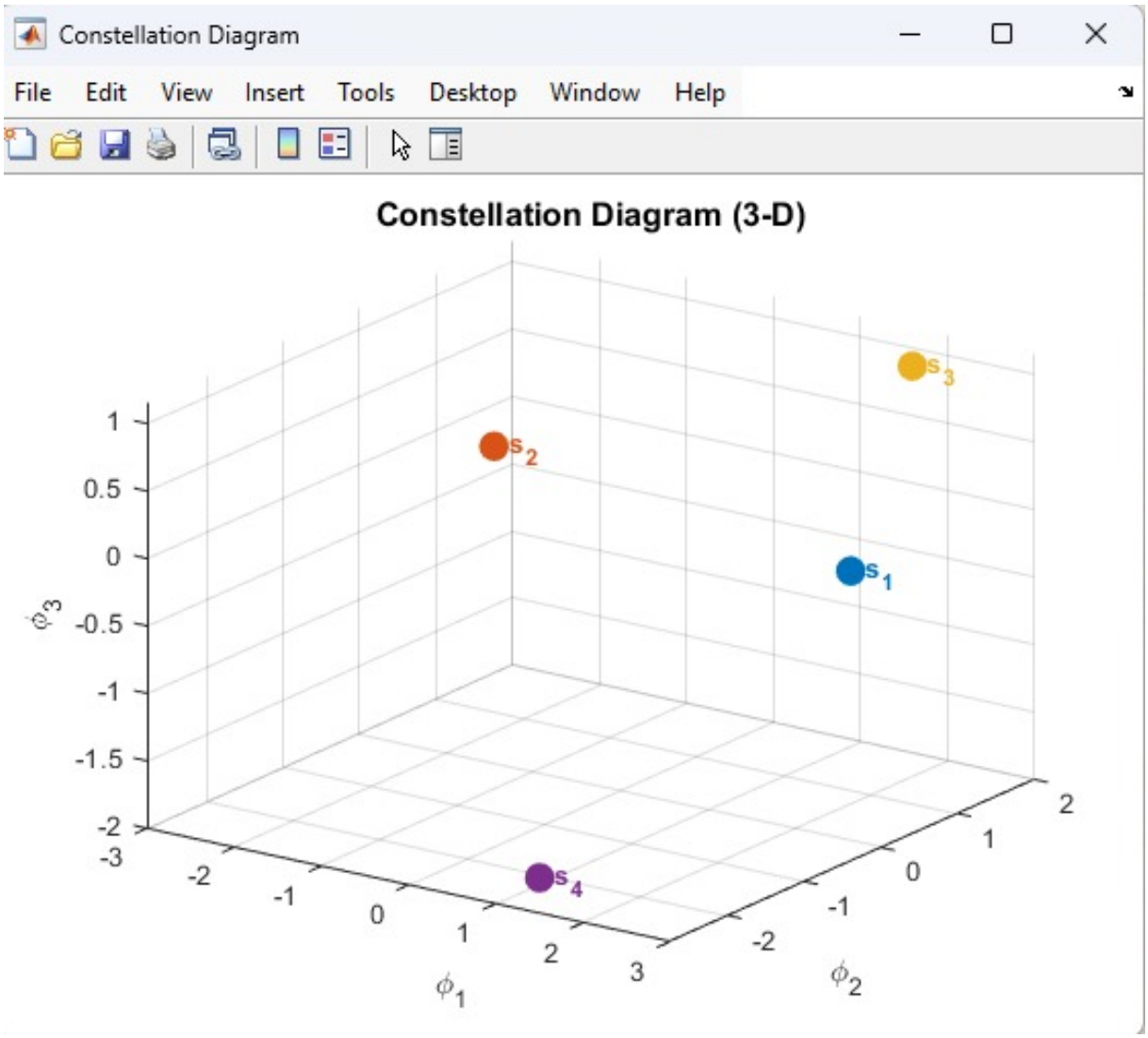


Figure 2: Constellation Diagram for Set 1

Signal Set 2

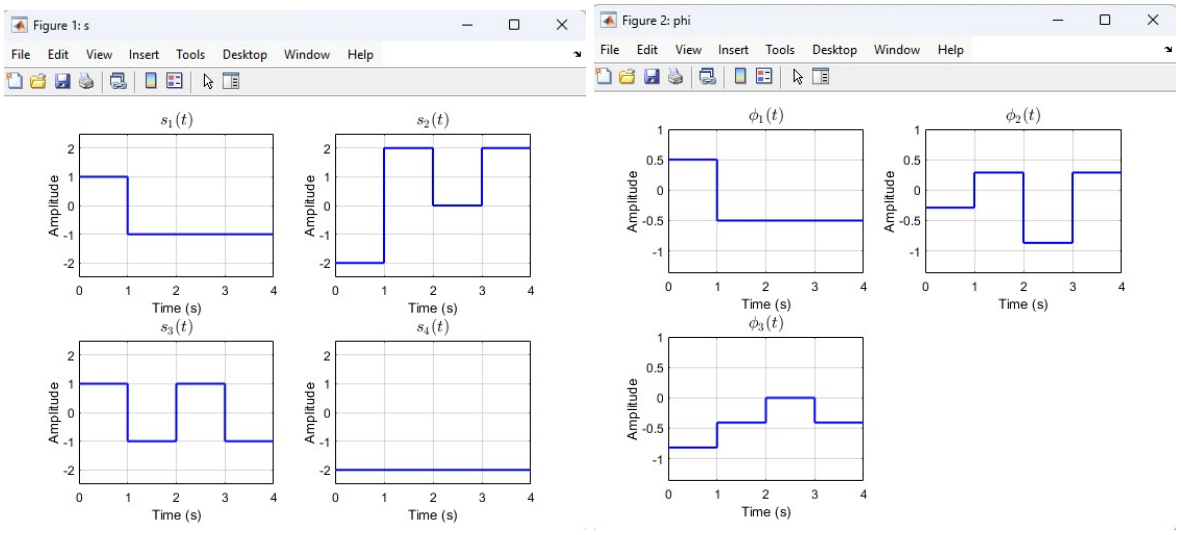
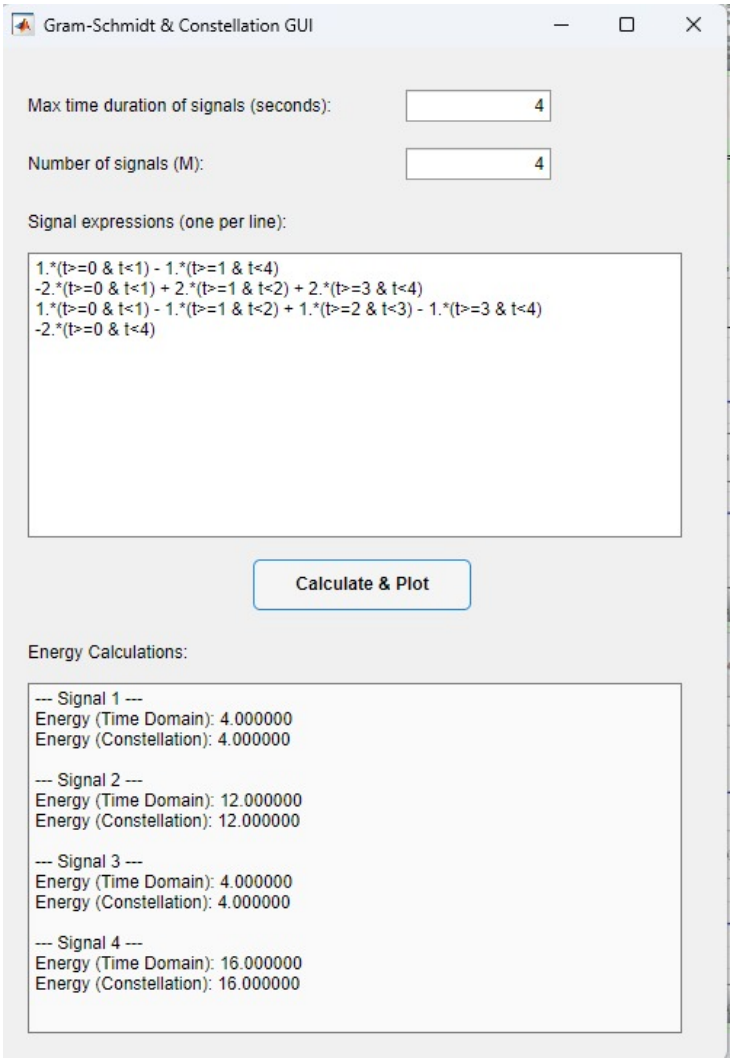


Figure 3: Input Signals and Gram-Schmidt Basis Functions for Set 2



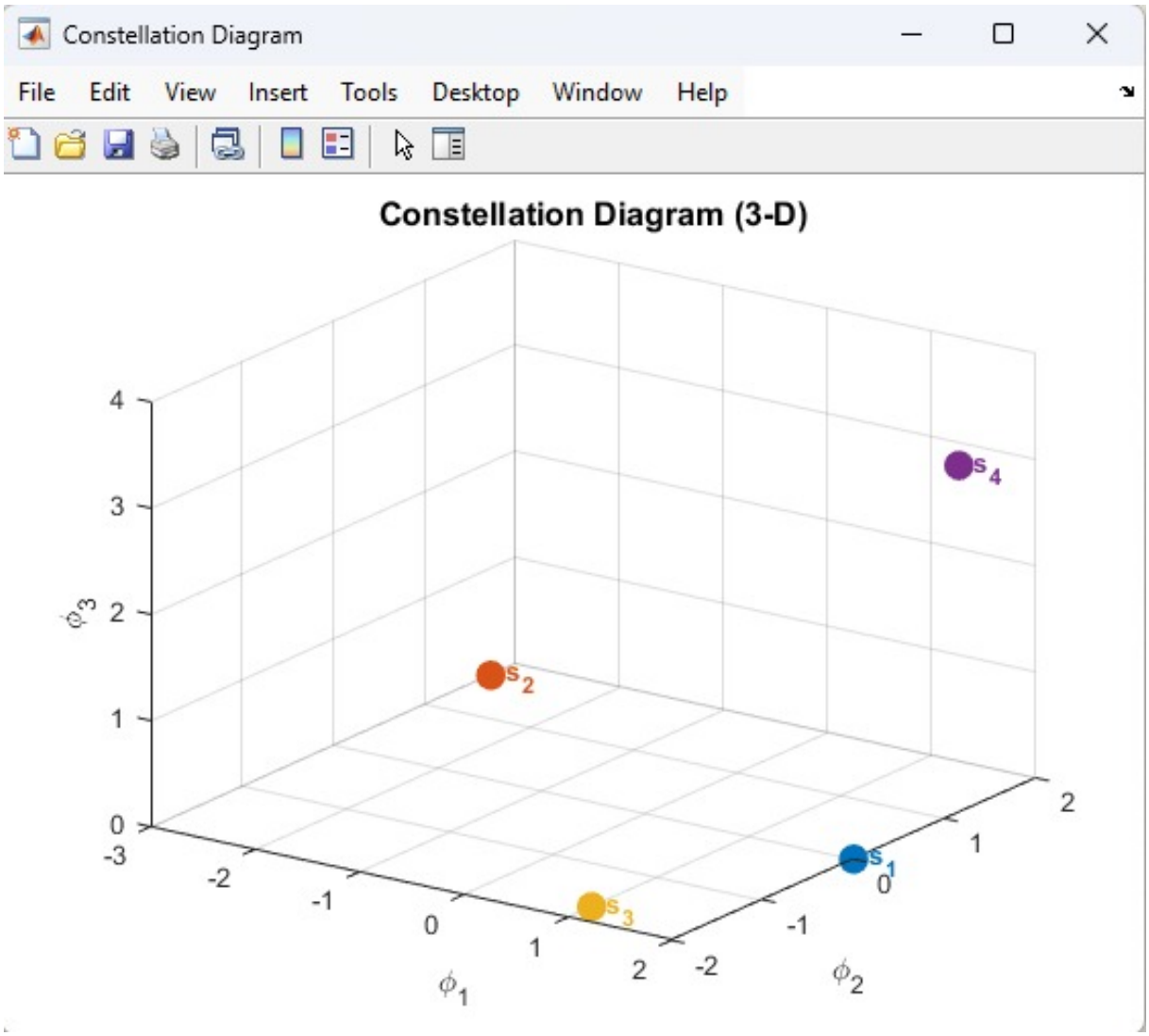


Figure 4: 3-D Constellation Diagram for Set 2

Signal Set 3

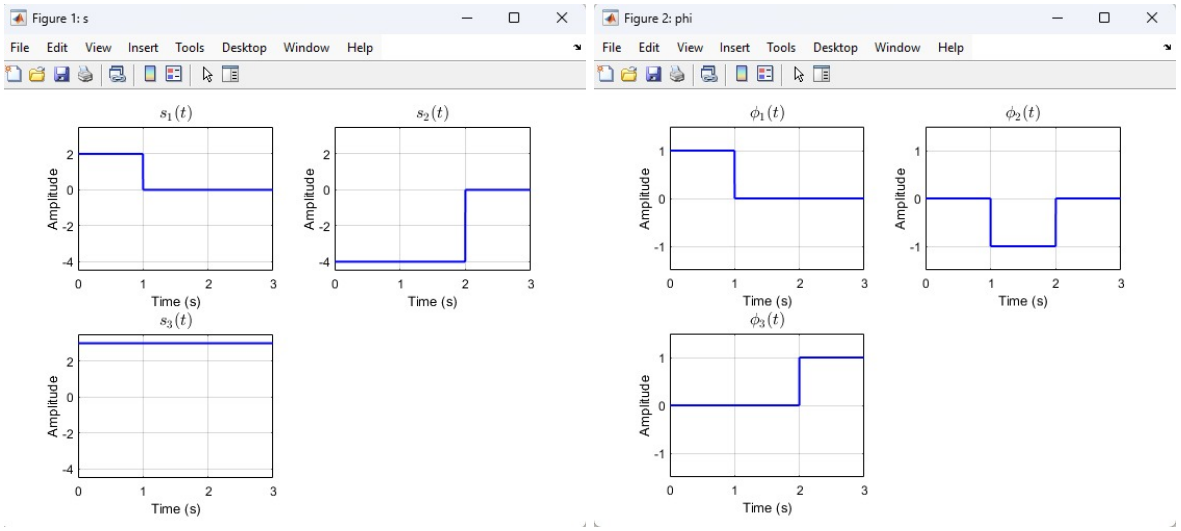
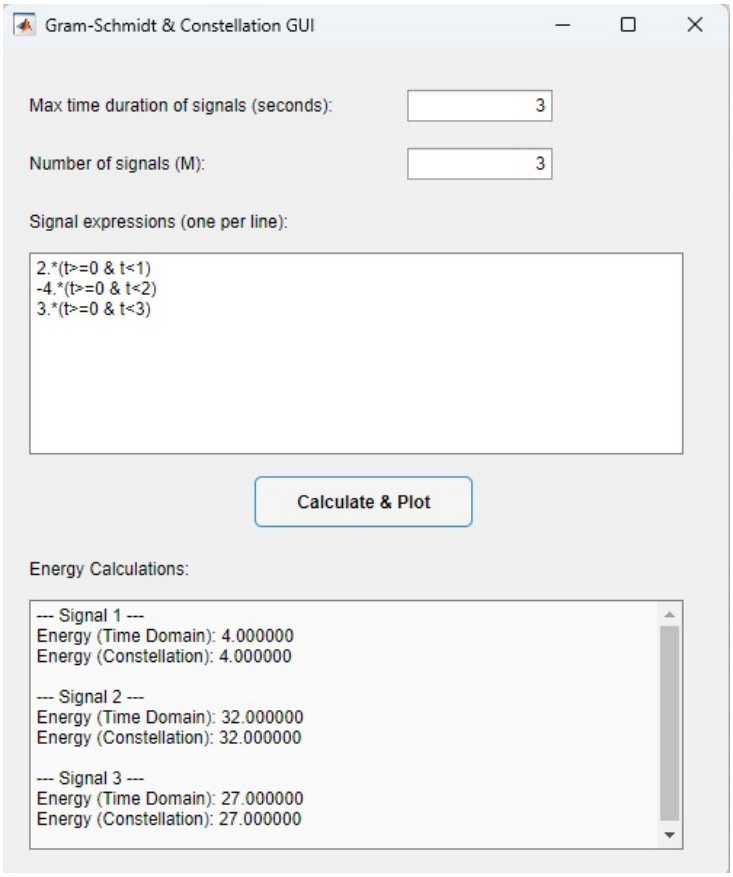


Figure 5: Input Signals and Gram-Schmidt Basis Functions for Set 3



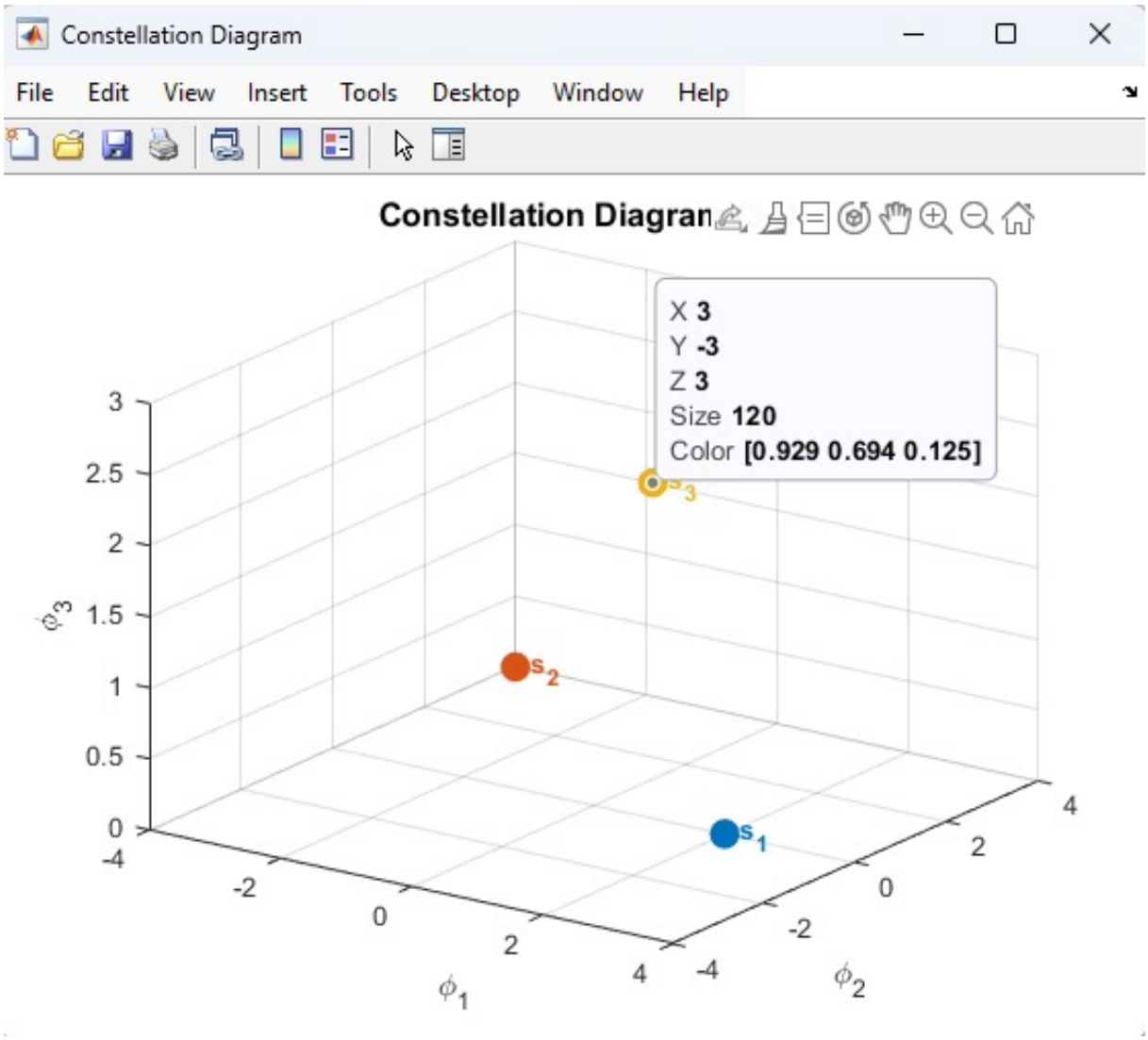


Figure 6: 3-D Constellation Diagram for Set 3

Part 2: Error Performance over AWGN Channel

Interactive GUI Integration

Similar to the first part of the project, the Graphical User Interface (GUI) for the AWGN channel simulation is fully embedded within the main MATLAB script. Rather than relying on a disconnected external application, the code sequentially triggers interactive dialogs. Through these integrated menus, the user can dynamically define the number of bits, the E_b/N_0 range, and select multiple line coding schemes simultaneously to run the BER simulation.

MATLAB Implementation

```
clc; clear; close all;

%%MINI GUI

prompt = {'Number of generated bits:', 'Eb/No Min (dB):', 'Eb/No Max (dB):', 'Eb/No Step (dB):'};
dlgtitle = 'Simulation Parameters';
dims = [1 55];
definput = {'100000', '-10', '10', '2'}; % Default values
answer = inputdlg(prompt, dlgtitle, dims, definput);

% Exit if user closes the window
if isempty(answer)
    disp('Simulation canceled by user.');
    return;
end

% Assign inputs to variables
num_bits = str2double(answer{1});
ebno_min = str2double(answer{2});
ebno_max = str2double(answer{3});
ebno_step = str2double(answer{4});

% Generate the Eb/No array from GUI inputs
EbNo_dB = ebno_min:ebno_step:ebno_max;
EbNo_lin = 10.^(EbNo_dB / 10);

% Prompt for Line Coding Selection
list = {'Unipolar NRZ', 'Manchester', 'Both'};
[indx, tf] = listdlg('PromptString', 'Select Line Coding:', ...
                     'SelectionMode', 'single', ...
                     'ListSize', [200, 80], ...
                     'ListString', list);

% Exit if user clicks cancel
if tf == 0
    disp('Simulation canceled by user.');
    return;
end

% Set boolean flags based on selection
do_nrz = (indx == 1 || indx == 3);
do_man = (indx == 2 || indx == 3);

%========================================
% Setup & Generation of Base Waveforms

ts = 0.001;
T_sym = 1;
n_per_sym = T_sym / ts;
t = 0:ts:T_sym - ts;
A = 1;

if do_nrz
    s0_unrz = zeros(1, n_per_sym);
```

```
    s1_unrz = A * ones(1, n_per_sym);
    matrix_unrz = [s1_unrz; s0_unrz];
    [basis_unrz, coeff_unrz] = gram_schmidt(matrix_unrz, ts);
    const_diag(coeff_unrz);
    title('Constellation Diagram: Unipolar NRZ');
end

if do_man
    half_n = n_per_sym / 2;
    s0_man = [-A*ones(1, half_n), A*ones(1, half_n)];
    s1_man = [A*ones(1, half_n), -A*ones(1, half_n)];
    matrix_man = [s1_man; s0_man];
    [basis_man, coeff_man] = gram_schmidt(matrix_man, ts);
    const_diag(coeff_man);
    title('Constellation Diagram: Manchester');
end

%========================================
% Generate Random Data & Encode

bits = randi([0, 1], 1, num_bits);

if do_nrz
    tx_unrz = zeros(num_bits, size(coeff_unrz, 2));
    tx_unrz(bits == 0, :) = repmat(coeff_unrz(2,:), sum(bits == 0), 1);
    tx_unrz(bits == 1, :) = repmat(coeff_unrz(1,:), sum(bits == 1), 1);
    Eb_unrz = mean(sum(coeff_unrz.^2,2));
    ber_unrz_sim = zeros(1, length(EbNo_dB));
    fig_rx_unrz = figure('Name', 'Received Constellation Diagram- Unipolar NRZ');
end

if do_man
    tx_man = zeros(num_bits, size(coeff_man, 2));
    tx_man(bits == 0, :) = repmat(coeff_man(2,:), sum(bits == 0), 1);
    tx_man(bits == 1, :) = repmat(coeff_man(1,:), sum(bits == 1), 1);
    Eb_man = mean(sum(coeff_man.^2,2));
    ber_man_sim = zeros(1, length(EbNo_dB));
    fig_rx_man = figure('Name', 'Received Constellation Diagram - Manchester');
end
% ========================================================================
%               AWGN Channel & Decoding

% Calculate dynamic subplot grid dimensions to prevent index errors
num_plots = length(EbNo_dB);
n_cols = ceil(sqrt(num_plots));
n_rows = ceil(num_plots / n_cols);

for k = 1:num_plots

    if do_nrz
        % Unipolar NRZ Channel
        N0_unrz = Eb_unrz / EbNo_lin(k);
        noise_unrz = sqrt(N0_unrz/2) * randn(size(tx_unrz));
        rx_unrz = tx_unrz + noise_unrz;

        % Plot Rx Constellation with DYNAMIC grid
        figure(fig_rx_unrz);
        subplot(n_rows, n_cols, k);
        scatter(rx_unrz, zeros(size(rx_unrz)), 1, 'filled', 'MarkerFaceAlpha', 0.5); hold
            on;
        % Plot the ideal points
        plot(coeff_unrz(:,1), [0 0], 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 5);
        xline(0, 'k--', 'LineWidth', 1);
        title(sprintf('Eb/No = %d dB', EbNo_dB(k)));

        % Decode Data (Minimum Euclidean Distance)
        dist0_unrz = sum((rx_unrz - coeff_unrz(2,:)).^2, 2);
        dist1_unrz = sum((rx_unrz - coeff_unrz(1,:)).^2, 2);
        rx_bits_unrz = zeros(1, num_bits);
        rx_bits_unrz(dist1_unrz < dist0_unrz) = 1;

        % Calculate Simulated BER
        ber_unrz_sim(k) = sum(rx_bits_unrz ~= bits) / num_bits;
```

```
    end

    if do_man
        % Manchester Channel
        N0_man = Eb_man / EbNo_lin(k);
        noise_man = sqrt(N0_man/2) * randn(size(tx_man));
        rx_man = tx_man + noise_man;

        % Plot Rx Constellation with DYNAMIC grid
        figure(fig_rx_man);
        subplot(n_rows, n_cols, k);
        scatter(rx_man, zeros(size(rx_man)), 1, 'filled', 'MarkerFaceAlpha', 0.5);  hold
            on;
        % Plot the ideal points
        plot(coeff_man(:,1), [0 0], 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 5);
        yticks([]);
        xline(0, 'k--', 'LineWidth', 1);
        title(sprintf('Eb/No = %d dB', EbNo_dB(k)));

        % Decode Data (Minimum Euclidean Distance)
        dist0_man = sum((rx_man - coeff_man(2,:)).^2, 2);
        dist1_man = sum((rx_man - coeff_man(1,:)).^2, 2);
        rx_bits_man = zeros(1, num_bits);
        rx_bits_man(dist1_man < dist0_man) = 1;

        % Calculate Simulated BER
        ber_man_sim(k) = sum(rx_bits_man ~= bits) / num_bits;
    end
end

% ========================================================================
%               Theoretical BER Calculation & Plotting

figure('Name', 'BER Performance Curves');

if do_nrz
    ber_unrz_th = qfunc(sqrt(EbNo_lin));
    semilogy(EbNo_dB, ber_unrz_sim, 'bo', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName
        ', 'Unipolar NRZ (Simulated)');
    hold on;
    semilogy(EbNo_dB, ber_unrz_th, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Unipolar NRZ (
        Theoretical)');
end

if do_man
    ber_man_th = qfunc(sqrt(2 * EbNo_lin));
    semilogy(EbNo_dB, ber_man_sim, 'rx', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName'
        , 'Manchester (Simulated)');
    hold on;
    semilogy(EbNo_dB, ber_man_th, 'r-', 'LineWidth', 1.5, 'DisplayName', 'Manchester (
        Theoretical)');
end

% Force the Y-axis to be logarithmic
set(gca, 'YScale', 'log');

grid on;
xlabel('E_b/N_o (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
title('Theoretical vs Practical BER', 'FontSize', 14);
legend('Location', 'southwest');

% ========================================================================
%                   ENERGY USING TIME DOMAIN FUNCTION
function E = energy(signal,ts)
    E = sum(abs(signal).^2)*ts;
end
% ========================================================================
%                    GRAM SCHMIDT FUNCTION
function [basis_functions,basis_coeff] = gram_schmidt(signals_matrix,ts)
    M = length(signals_matrix(:,1));
    L = length(signals_matrix);
    basis_functions = zeros(M,L);
```

```
    basis_coeff = zeros(M);
    E_s1 = energy(signals_matrix(1,:),ts);
    basis_functions(1,:) = signals_matrix(1,:)/sqrt(E_s1);
    basis_coeff(1) = sqrt(E_s1);
    N = 1;
    for i = 2:M
        for j = 1:N
            basis_coeff(i,j) = sum(basis_functions(j,:).*signals_matrix(i,:))*ts;
        end
        g_i = signals_matrix(i,:) - basis_coeff(i,:)*basis_functions;
        if(max(abs(g_i)) < 1e-10)
            continue;
        else
            E_gi = energy(g_i,ts);
            basis_functions(i,:) = g_i/sqrt(E_gi);
            N = N+1;
       end
    end
    basis_functions = basis_functions(1:N,:);
    basis_coeff = basis_coeff(:,1:N);
end
% ========================================================================
%                  PLOTTING CONSTELLATION
function const_diag(basis_coeff)
    [M, N] = size(basis_coeff);
    figure('Name', 'Constellation Diagram', 'NumberTitle','off', 'Color',[1 1 1]);
    colors_c = lines(M);
    hold on; grid on;

    if N == 1
        for m = 1 : M
            scatter(basis_coeff(m,1), 0, 120, colors_c(m,:), 'filled');
            text(basis_coeff(m,1), 0.08, sprintf('s_{%d}',m), 'HorizontalAlignment', '
                center', 'Color', colors_c(m,:), 'FontWeight', 'bold');
        end
        xlabel('\phi_1'); ylim([-1 1]);
        xline(0,'k--');
    elseif N == 2
        for m = 1 : M
            scatter(basis_coeff(m,1), basis_coeff(m,2), 120, colors_c(m,:), 'filled');
            text(basis_coeff(m,1) + 0.05, basis_coeff(m,2) + 0.05, sprintf('s_{%d}',m), '
                Color', colors_c(m,:), 'FontWeight', 'bold');
        end
        xlabel('\phi_1'); ylabel('\phi_2');
        xline(0,'k--'); yline(0,'k--');
    else
        for m = 1 : M
            scatter3(basis_coeff(m,1), basis_coeff(m,2), basis_coeff(m,3), 120, colors_c(
                m,:), 'filled');
            text(basis_coeff(m,1), basis_coeff(m,2), basis_coeff(m,3), sprintf('  s_{%d}'
                ,m), 'Color', colors_c(m,:), 'FontWeight', 'bold');
        end
        xlabel('\phi_1'); ylabel('\phi_2'); zlabel('\phi_3');
        view(35, 25);
    end
    title(sprintf('Constellation Diagram (%d-D)', min(N, 3)), 'FontSize', 12);
end
```

Theoretical Derivations

The probability of bit error (P_e) for an optimal correlation receiver is:

$$P_e = Q\left(\frac{d}{\sqrt{2N_0}}\right)$$

1. Unipolar NRZ

For Unipolar NRZ, the Euclidean distance between symbols is $d = \sqrt{2E_b}$. Substituting d:

$$P_e = Q\left(\sqrt{\frac{E_b}{N_0}}\right)$$

2. Manchester Encoding

For Manchester, signaling is antipodal, meaning $d = 2\sqrt{E_b}$. Substituting d:

$$P_e = Q\left(\sqrt{\frac{2E_b}{N_0}}\right)$$

Simulation Results and comments

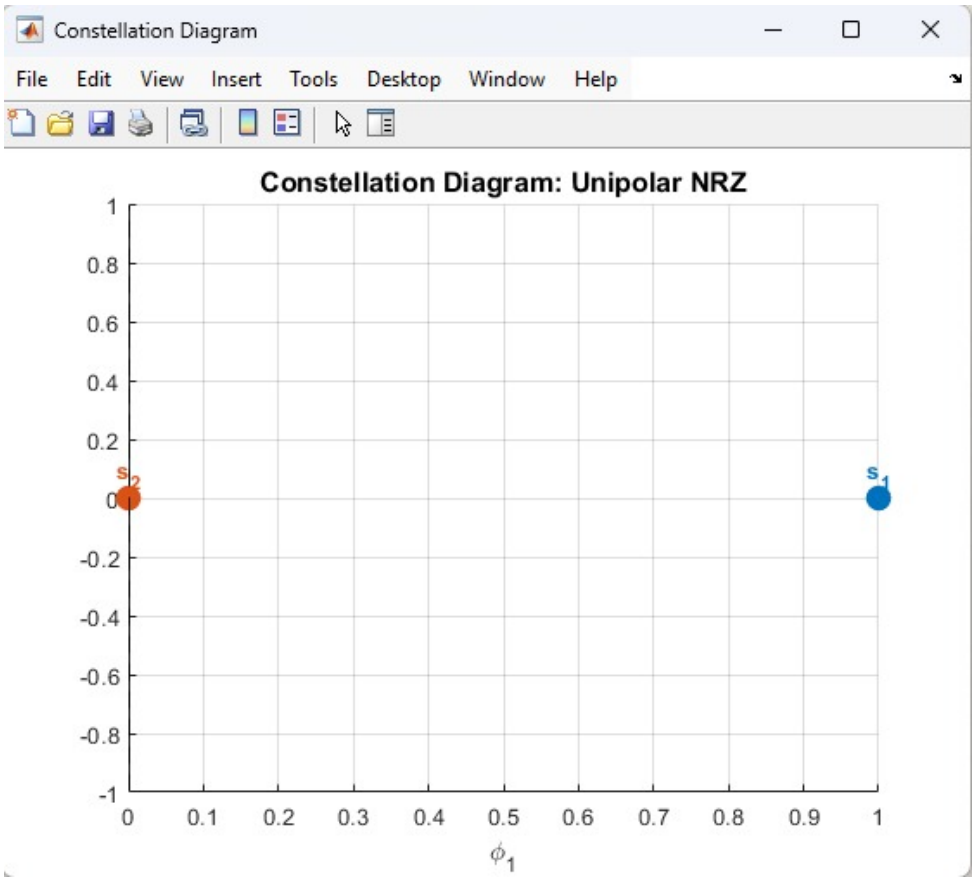


Figure 7: Constellation Diagram: Unipolar NRZ

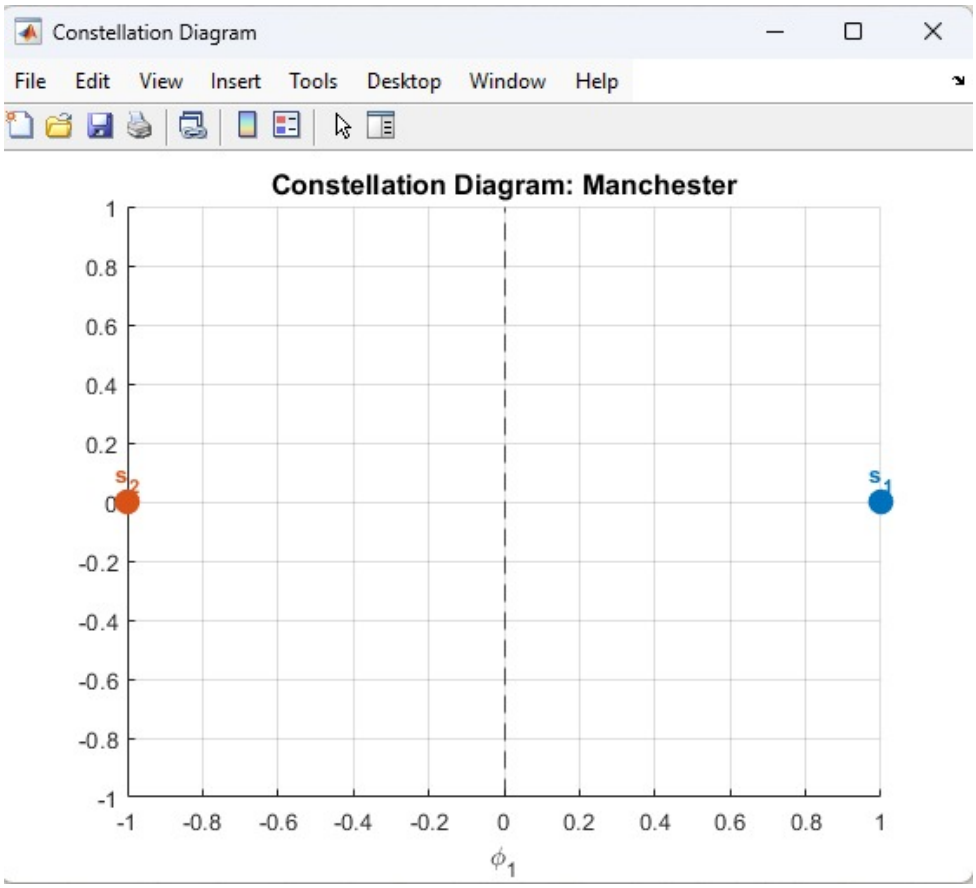


Figure 8: Constellation Diagram: Manchester

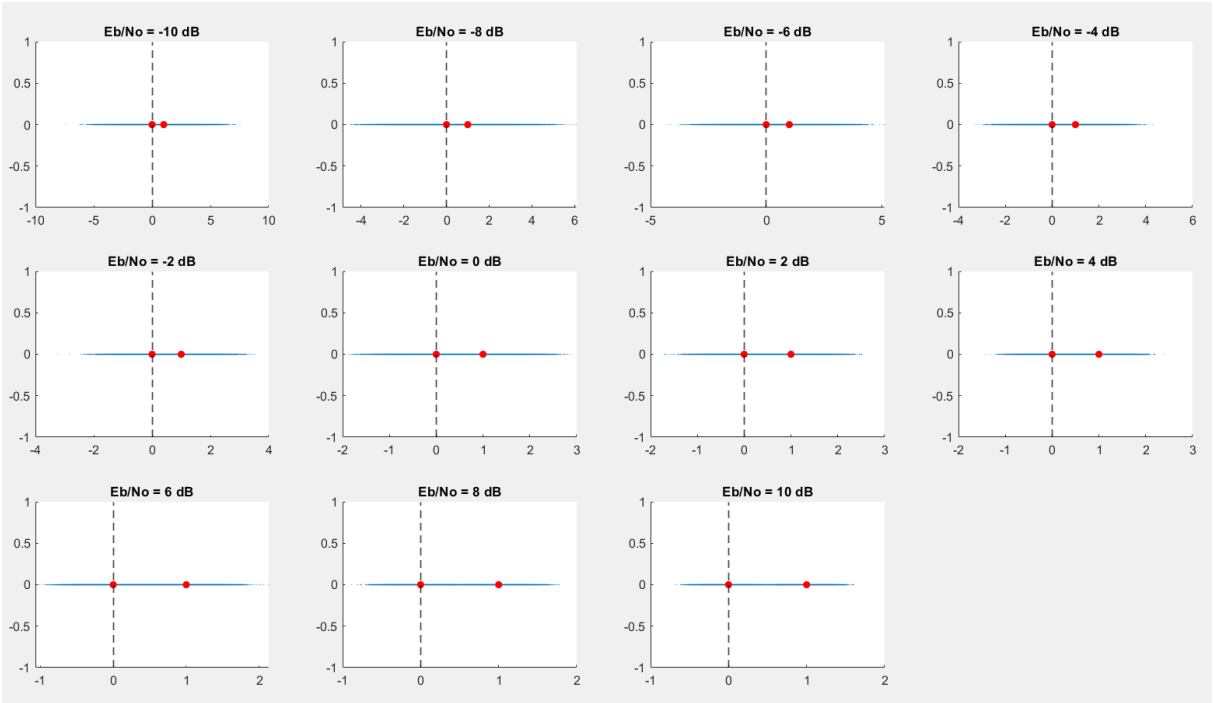


Figure 9: Received Noisy Constellation: Unipolar NRZ

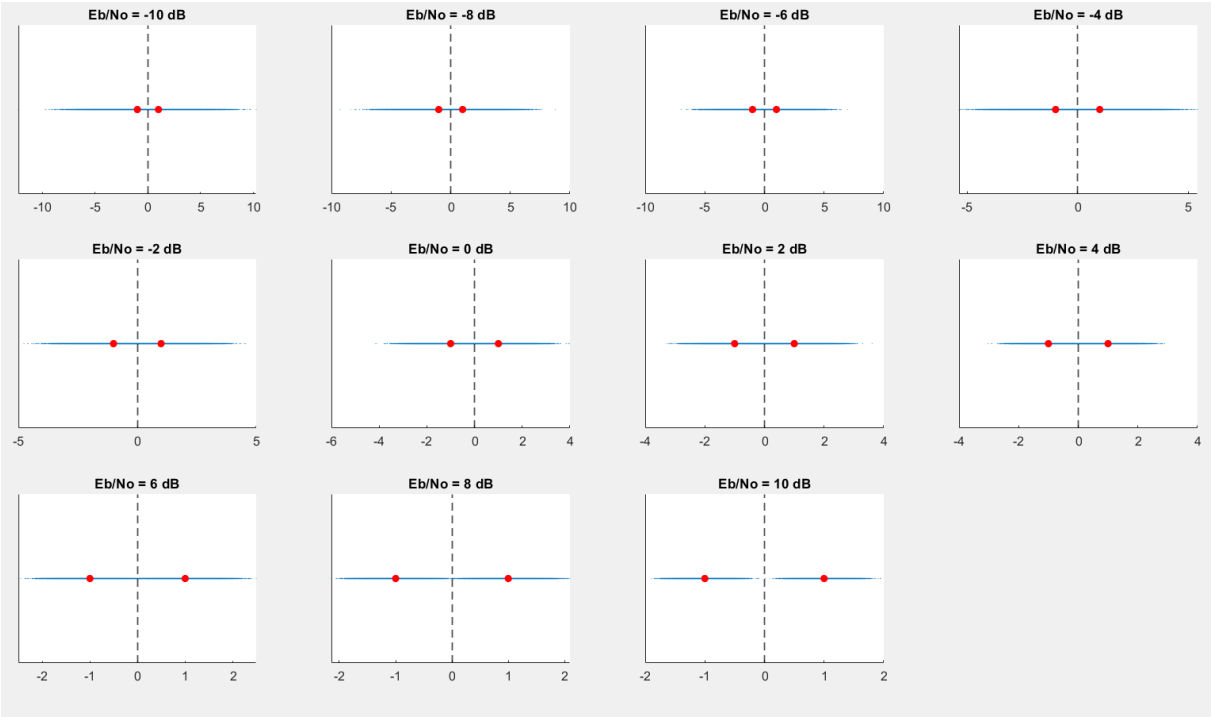


Figure 10: Received Noisy Constellation: Manchester

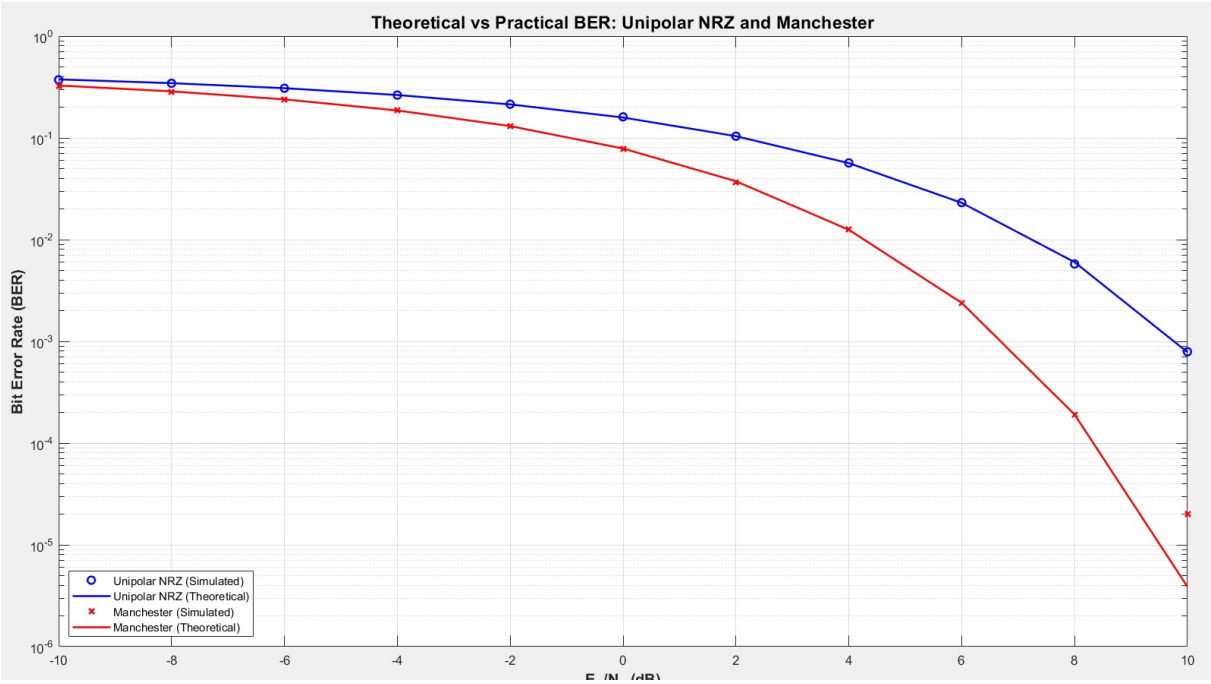


Figure 11: Theoretical vs Practical BER: Unipolar NRZ and Manchester

Commentary on the Results

As shown in the BER performance figure, the simulated data points (blue circles for Unipolar NRZ and red crosses for Manchester) perfectly track their respective theoretical curves (solid lines). It is visually evident that Manchester Encoding significantly outperforms Unipolar NRZ in the AWGN channel. This performance gap exists because Manchester is an antipodal signaling scheme ($s_1 = -s_0$), which maximizes the Euclidean distance between symbols for a given average bit energy compared to the on-off nature of Unipolar NRZ.